

1) Un programma e' eseguito su una CPU con frequenza di 2.5 GHz.
 Il programma necessita di 9×10^9 istruzioni con il compilatore corrente, e il CPI medio è 1.6.
 Un nuovo compilatore riduce il numero di istruzioni al 65%, tuttavia il CPI cambia di 0.9.
 La frequenza della CPU rimane invariata.
 Di quanto sarà più veloce il programma con il nuovo compilatore?

☐ a. 3.45
☒ b. 1.71
☐ c. 2.62
☐ d. 4.22
☐ e. 1.1

RICORDIAMO :

$$P_R = \frac{1}{T_E}$$

PRESTAZIONI

$$T_{CPU} = N^{\circ} \text{CICLI} \cdot T$$

TEMPO DI CPU RELATIVO AD UN PROGRAMMA

$$N^{\circ} \text{CICLI} = I \cdot \text{CPI}$$

N° ISTRUZIONI DEL PROGRAMMA

NUMERO MEDIO DI CICLI DI CLOCK PER ISTRUZIONE

$$T = \frac{1}{f}$$

PERIODO O INVERSO DELLA FREQUENZA (SEC, NSEC)

$$f = \frac{1}{T}$$

FREQUENZA DI CLOCK (Hz, GHz)

$$T_{CPU} = \frac{N^{\circ} \text{CICLI}}{f}$$

QUINDI :

$f = 2,5 \text{ GHz} = 2,5 \cdot 10^9 \text{ Hz}$ ← RICORDIAMO DI PORTARE IN Hz

$I_1 = 9 \cdot 10^9 \quad \text{CPI}_1 = 1,6$

$I_2 = \frac{9 \cdot 10^9 \cdot 65}{100} = 585 \cdot 10^7 = 5,85 \cdot 10^9 \quad \text{CPI}_2 = 0,9 \cdot 1,6 = 1,44$

$f_1 = 2,5 \cdot 10^9 \text{ Hz}$

$T_{CPU1} = \frac{9 \cdot 10^9 \cdot 1,6}{2,5 \cdot 10^9} = 5,76 \text{ SEC}$ $T_{CPU2} = \frac{585 \cdot 10^7 \cdot 1,44}{2,5 \cdot 10^9} = 3,37 \text{ SEC}$

$\frac{P_{R2}}{P_{R1}} = \frac{\frac{1}{T_{CPU2}}}{\frac{1}{T_{CPU1}}} = \frac{T_{CPU1}}{T_{CPU2}} = \frac{5,76 \text{ SEC}}{3,37 \text{ SEC}} = 1,709 \approx 1,71$

2) Converti il numero binario 1101011 in decimale.

☐ a. 48
☐ b. 271
☐ c. 573
☒ d. 107

2 METODI :

1) SI APPLICA LA DEFINIZIONE DI SISTEMA DI NUMERAZIONE POSIZIONALE:

$1 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = (64 + 32 + 8 + 2 + 1)_{10} = 107_{10}$

2) PARTENDO DALLA CIFRA + SIGNIFICATIVA, MOLTIPLICO PER 2 LA SOMMA PARZIALE (INIZIALMENTE UGUALE AD 1) E AGGIUNGO LA CIFRA BINARIA IN CUI MI TROVO (MENO COMPRESSO PER UN CALCOLATORE):

$1 \rightarrow (0 \cdot 2) + 1 = 1 \quad | \quad 1 \rightarrow (6 \cdot 2) + 1 = 13$

$$\begin{array}{l}
 1 \rightarrow (1 \cdot 2) + 1 = 3 \\
 0 \rightarrow (3 \cdot 2) + 0 = 6 \\
 1 \rightarrow (13 \cdot 2) + 0 = 26 \\
 1 \rightarrow (26 \cdot 2) + 1 = 53 \\
 1 \rightarrow (53 \cdot 2) + 1 = 107
 \end{array}
 \quad
 1101011_2 = 107_{10}$$

Converti 351 in binario.

- ☒ a. 101011111
☐ b. 110011111
☐ c. 101101111
☐ d. 100111111

2 METODI:

1) SOTTRAGGO AL NUMERO DECIMALE LA +GRANDE POTENZA DI 2 NON PIÙ GRANDE DEL NUMERO STESSO FINCHÉ NON RAGGIUNGO 0.

$$\begin{array}{l}
 351 - 256 = 95 \quad 95 - 64 = 31 \quad 31 - 16 = 15 \\
 \quad \quad \quad 2^8 \quad \quad \quad 2^6 \quad \quad \quad 2^4 \\
 15 - 8 = 7 \quad 7 - 4 = 3 \quad 3 - 2 = 1 \quad 1 - 1 = 0 \\
 \quad \quad \quad 2^3 \quad \quad \quad 2^2 \quad \quad \quad 2^1 \quad \quad \quad 2^0
 \end{array}$$

$$351_{10} = 101011111_2$$

2) DIVIDERE RIPETUTAMENTE IL NUMERO DECIMALE PER 2, DOVE IL RESTO DELLA DIVISIONE È IL RISULTATO IN BASE 2 E IL QUOZIENTE VIENE PRESO COME NUOVO NUMERO DECIMALE DA DIVIDERE FINCHÉ NON SI RAGGIUNGE 0:

	Q	R
351	175	1
175	87	1
87	43	1
43	21	1
21	10	1
10	5	0
5	2	1
2	1	0
1	0	1

LO SI LEGGE AL CONTRARIO

$$\text{QUINDI: } 351_{10} = 101011111_2$$

Converti 423 in esadecimale.

- ☐ a. AB2
☐ b. FF1
☒ c. 1A7
☐ d. 61A

LA STESSA COSA VALE IN ESADECIMALE, VEDIAMO IL METODO 2:

	Q	R
423	26	7
26	1	A
1	0	1

$$423 : 16 = 26,4375$$

$$0,4375 \cdot 16 = 7$$

RICORNIAMO IL SISTEMA ESADECIMALE:

$$\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F\}$$

$$423_{10} = 1A7_{16}$$

Converti 384.4692 in binario

- ☒ a. 110000000.01111000
☐ b. 110000000.11110000
☐ c. 101111111.11110000
☐ d. 110000001.1110000

LA PARTE INTERA SI CONVERTE IN BINARIO COME ABBIAMO GIÀ VISTO:

	Q	R
384	192	0
192	96	0
96	48	0
48	24	0
24	12	0
12	6	0
6	3	0
3	1	1
1	0	1

$$384_{10} = 110000000_2$$

PER LA PARTE FRAZIONARIA INVECE, COMUNEMENTE CHIAMATA **NFR** ABBIAMO UN METODO DIVERSO:

SI MOLTIPLICA RIPETUTAMENTE L'NFR PER 2, COSÌ OTTENENDO UNA **PARTE INTERA** CHE SARÀ LA CIFRA FRAZIONARIA IN BASE 2, E UNA **PARTE FRAZIONARIA** CHE AVENTERÀ IL **NUOVO NFR**.

CI SI FERMA QUANDO LA PARTE FRAZIONARIA RAGGIUNGE 0, OPPURE SE PERIODICA SI STABILISCE UNA **PRECISIONE** → SE SI VOLE UNA PRECISIONE DI 10^{-2} ALLORA CI VORRANNO 7 BIT DATO CHE $2^{-6} < 10^{-2} < 2^{-7}$

NFR

BIT

QUINDI:

$$0.4692 \cdot 2 = 0.9384 \rightarrow 0$$

$$0.9384 \cdot 2 = 1.8768 \rightarrow 1$$

$$0.8768 \cdot 2 = 1.7536 \rightarrow 1$$

$$0.7536 \cdot 2 = 1.5072 \rightarrow 1$$

$$0.5072 \cdot 2 = 1.0144 \rightarrow 1$$

$$0.0144 \cdot 2 = 0.0288 \rightarrow 0$$

$$0.0288 \cdot 2 = 0.0576 \rightarrow 0$$

$$0.0576 \cdot 2 = 0.1152 \rightarrow 0$$

$$384.4692 = 110000000.01111000_2$$

... È PERIODICO (IL PROBLEMA È CHE LA PARTE PERIODICA È MOLTO AVANTI QUINDI MOSTRIAMO SOLO IL **PRE-PERIODICO**)
↓
CI DOVREBBE DARE UNA **PRECISIONE**.

Somma 1011011 + 1101101.

- ☐ a. 11000100
- ☐ b. 10101110
- ☒ c. 11001000
- ☐ d. 11101000

QUI DOBBIAMO ESEGUIRE LA SOMMA DI 2 **NUMERI BINARI SENZA SEGNO**:

GIUSTAMENTE ANCHE IN BASE 2 POTREMO AVER BISOGNO DI UN **RIPORTO (CARRY)** PER LA SOMMA (E UN **PRESTITO** PER LA SOTTRAZIONE)

IL PESO DEL **RIPORTO / PRESTITO** È 2 IN BASE 2.

$$\begin{aligned} 1+0 &= 1 & 0+1 &= 1 & \text{CON RIPORTO} &= 0 \\ 0+0 &= 0 & & & \text{CON RIPORTO} &= 0 \\ 1+1 &= 0 & & & \text{CON RIPORTO} &= 1 \end{aligned}$$

$$\begin{array}{r} 1111111 \\ 1011011 \\ 1101101 \\ \hline 11001000 \end{array}$$

Sottrai 3A5F a 75AB.

- ☒ a. 3B4C
- ☐ b. A317
- ☐ c. BF3B
- ☐ d. F4D2

LO STESSO VALE PER LE ALTRE BASI:

$$\begin{array}{r} \tilde{7}5\tilde{A}B \\ 3A5F \\ \hline 3B4C \end{array} \quad \begin{array}{r} 11+16 = 27 \\ 27 - \\ 18 = \\ 12 \rightarrow C \end{array}$$

$$\begin{array}{r} (5+16) = 21 \\ 21 - 10 = 11 \end{array} \quad \begin{array}{r} \tilde{A} = (10-1) = 9 \\ 9 - \\ 8 = \\ 1 \end{array}$$

Risolve il seguente esercizio in Modulo e Segno:

$$-85 + (+60)$$

- ☒ a. 10011001
- ☐ b. 00011001
- ☐ c. Non rappresentabile (overflow)
- ☐ d. 10011011

QUI DOBBIAMO RISOLVERE UNA SOMMA IN **MODULO E SEGNO (M&S)**:

PER RAPPRESENTARE **NUMERI BINARI CON SEGNO** POSSIAMO UTILIZZARE LA **RAPPRESENTAZIONE MODULO E SEGNO**.

QUESTA RAPPRESENTAZIONE UTILIZZA 1 BIT (IL + SIGNIFICATIVO) PER IL **SEGNO** (0 = POSITIVO, 1 = NEGATIVO) E I RESTANTI $n-1$ BIT PER RAPPRESENTARE IL **MODULO (VALORE ASSOLUTO)** DEL NUMERO.

IL **RANGE** CHE **M&S** PUÒ RAPPRESENTARE È $[-(2^{n-1}-1), +(2^{n-1}-1)]$, POICHÉ LO **ZERO** VIENE RAPPRESENTATO 2 VOLTE.

LE REGOLE DA RISPETTARE PER ESEGUIRE LE OPERAZIONI IN M&S SONO:

- **SOMMA** : $(N_1 + N_2)$

- **SEGNI CONCORDI**: **SEGNO** = UGUALE, **MODULO** = SOMMA DEI 2 MODULI.

- **SEGNI DISCORDI**: **SEGNO** = UGUALE ALL'OPERANDO PIÙ **MODULO** + **GRANDE**,
MODULO = DIFFERENZA TRA IL **MODULO** + **GRANDE** E IL **MODULO** + **PICCOLO**.

- SOTTAZIONE: $(N_1 - N_2)$

SOTTAPAZIONE: $(N_1 - N_2)$

- SEGNI CONCORDI: $\rightarrow$ ENTRAMBI POSITIVI: SEGNO = + SE $|N_1| \geq |N_2|$, SENNO -.

SI INTENDE IL MODULO.

ENTRABI NEGATIVI: SEGNO = - SE $|N_1| > |N_2|$, SENNO +.

MODULO (ENTRABI): DIFFERENZA TRA IL MODULO + GRANDE E IL MODULO + PICCOLO

- **SEGNI DISCORDI**: **SEGNO** = UGUALE A QUELLO NEL MINUENDO
MODULO = SOMMA DEI 2 MODULI ($|N_1| + |N_2|$)

→ VADO OLIVE $+ (2^{n-1} - 1)$

QUANDO SI PUÒ VERIFICARE OVERFLOW? CON SOMMA DI NUMERI CONCORRANTI E DIFFERENZA DI NUMERI DISCORRANTI.

↳ VADO OUTRE $-(2^{h-1}-1)$

Quindi:

$$-85 + (+60) \rightarrow \text{SOMMA DEI DISCORDI} \rightarrow \text{SEGNO} = - \quad (85 > 60)$$

$$\text{MODULO} = (85 - 60) = 25$$

RISULTATO = -25, L'ESERCIZIO SEMBRA RICHIEDERE $n = 8$ BIT

QUINDI -25_{10} IL M&S EQUIVALE AD: $1 \mid 0011001$
 OPPURE: $\downarrow$ 25

OPPURE:

OPPURE: $-85 = 11010101$ (+GRANDE) $+60 = 00111100$ (-)
DISCORDI

$$\begin{array}{r} \tilde{1} \tilde{1} \tilde{0} \tilde{1} 0 1 0 1 - (|N_1| - |N_2|) \\ 0 1 1 1 1 0 0 \\ \hline 0 0 1 1 0 0 1 \end{array}$$

Risolvi la seguente sottrazione in Modulo e Segno:

$$-120-(+10)$$

- ☒ a. Non rappresentabile (overflow)

☐ b. 11111010

☐ c. 11000001

☐ d. 10110100

$-120 = \overset{\text{MINUS 10}}{11111000} + 10 = 00001010$

$\swarrow \quad \searrow$
DISCORD!

SEGNO : -
MODULO : $(N_1 + N_2)$

1 1 1 1 1 0 0 0 +
0 0 0 1 0 1 0

0 0 0 0 0 1 0

OVERFLOW

COME ACCORGERSI SUBITO?

Se $n = 8$ BIT IL RANGE È $[-127, +127] \rightarrow -130$ È OLTRE IL RANGE

Converti in complemento a 2 il seguente numero:

9

~~a.~~ 10111

☐ b. 11001

☐ c. 10110

☐ d. 11000

IL COMPLEMENTO A 2 (C_2) È UN'ALTRA MODALITÀ DI RAPPRESENTAZIONE DEI NUMERI CON SEGNO.

C1 SONO 3 MODI PER RAPPRESENTARE UN NUMERO IN C2:

1) PONIAMO COME n IL NUMERO DI BIT DEL VALORE DA RAPPRESENTARE IN C2:

$$\bar{N} = 2^n - N$$

NUMERO IN C2 NUMERO IN BINARIO (VALORE ASSOLUTO)

2) $\bar{N} = (2^n - 1) - N \Rightarrow \bar{\bar{N}} = \bar{N} + 1$ IL COMPLEMENTO A 1 DI UN NUMERO INVERTE GLI 0 CON GLI 1 E VICEVERSA.

COMPLEMENTO A 1

3) PARTO DALLA CIFRA MENO SIGNIFICATIVA DEL NUMERO, E CONVERTO IN COMPLEMENTO A 1 TUTTE LE CIFRE FINO AL PRIMO 1 INCLUSO.

NUMERI POSITIVI: $+N \rightarrow 0 |N|$ (COME IN MODULO E SEGNO)

NUMERI NEGATIVI: $-N \rightarrow 1 |\bar{N}|$

IL RANGE CHE PUÒ RAPPRESENTARE È: $[-(2^{n-1}), +(2^{n-1}-1)]$

LO ZERO VIENE RAPPRESENTATO UNA VOLTA ED È: $00...00$, SEMPRE $10...00 \bar{E} -2^{n-1}$.

QUINDI CONVERTI 9 IN COMPLEMENTO A 2 SIGNIFICA TROVARE -9:

$n = 5$ PERCHÉ IL RANGE È $[-16, +15]$ CON $n = 4 \Rightarrow [-8, +7]$

$$N = 01001 \quad \bar{N} = 10110 \quad \bar{\bar{N}} = 10111$$

AGGIUNGO UNO 0
PER ARRIVARE A 5 BIT

OPPURE: $1 |\bar{N}| \rightarrow N = 1001 \quad \bar{N} = 0110 \quad \bar{\bar{N}} = 0111$

$\hookrightarrow 10111$

Converti in complemento a 2 il seguente numero:

12

- ☐ a. 11010
- ☒ b. 10100
- ☐ c. 10111
- ☐ d. 11011

IL RANGE $[-16, 15]$ MI BASTA QUINDI $n = 5$

$$N = 01100 \quad \bar{N} = 10011 \quad \bar{\bar{N}} = 10100$$

Somma in complemento a 2 su 5 bit:

$+7 + (-5)$

- ☒ a. 00010
- ☐ b. 10010
- ☐ c. 01010
- ☐ d. 00001

LA SVOLTA NEL C2 RISALE AL FATTO CHE LE OPERAZIONI SONO SEMPLICISSIME:

SOMMA $\rightarrow N_1 + N_2$

SOTTRAZIONE $\rightarrow N_1 - N_2 = N_1 + (-N_2) = N_1 + \bar{\bar{N}}_2$

QUINDI: $n = 5$ $+7 + (-5) = N_1 + \bar{N}_2$ $N_1 = 00111$ $N_2 = 00101$
 $\bar{N}_2 = 11010$ $\bar{N}_2 = 11011$

$$\begin{array}{r} 1 \quad 1 \quad 1 \quad 1 \\ 0 \quad 0 \quad 1 \quad 1 \quad 1 \\ 1 \quad 1 \quad 0 \quad 1 \quad 1 \\ \hline 0 \quad 0 \quad 0 \quad 1 \quad 0 \end{array} + =$$

L'ULTIMO CARRY SI SCARTA

È SEMPLICISSIMO NOTARE L'OVERFLOW, BASTA VEDERE SE L'ULTIMO CARRY (QUELLO CHE SI SCARTA) È DIVERSO DAL PENULTIMO CARRY, ALTRIMENTI NIENTE OVERFLOW.

Converti il seguente numero in singola precisione IEEE754:

-18.375

- ☒ a. 1 10000011 001001100000000000000000
☐ b. 1 10000111 001010100000000000000000
☐ c. 1 10000001 111001100000000000000000
☐ d. 1 10000010 001011000000000000000000

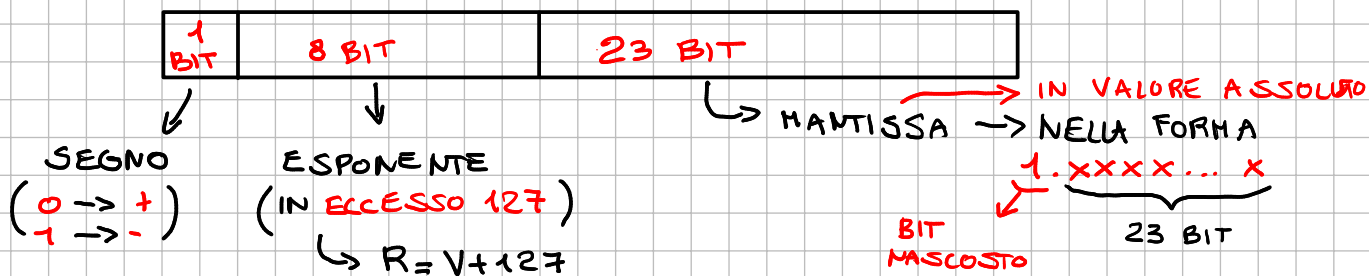
LO STANDARD IEEE754 STABILISCE IL FORMATO DI RAPPRESENTAZIONE DEI NUMERI A VIRGOLA MOBILE, LE OPERAZIONI DI BASE, LE CONVERSIONI E LE CONDIZIONI ECCEZIONALI.

DEFINISCE 3 FORMATI DI PRECISIONE:

- SINGOLA (32 BIT)
- DOPIA (64 BIT)
- QUADRUPLA (128 BIT)

LA SINGOLA PRECISIONE:

È COME UN M&S MA CON I FLOATING POINT.



QUINDI: -18.375 IN SINGOLA PRECISIONE IEEE754

- 1) SEGNO → 1
- 2) CONVERTIRE IN BINARIO $-18.375 \rightarrow 10010.011$ 0,375
- 3) NORMALIZZARE IL NUMERO $1.0010011 \cdot 2^4$ (2⁴ PERCHÉ HO SPOSTA LA VIRGOLA DI 4 VERSO SINISTRA)
- 4) CALCOLARE L'ESPOONENTE SU 8 BIT CON ECCESSO 127:

$$R = 4 + 127 = 131_{(10)} \rightarrow 10000011_2$$

QUINDI LA RAPPRESENTAZIONE SAREBBE: 1 | 10000011 | 0010011000...00

Somma questi due numeri in singola precisione IEEE754:

$$-6.5 + (-2.25)$$

- ☒ a. 1 10000010 000110000000000000000000
- ☐ b. 1 10000011 001100000000000000000000
- ☐ c. 1 10000100 000100000000000000000000
- ☐ d. 1 10000011 011100000000000000000000

PER ESEGUIRE OPERAZIONI IN **FLOATING POINT**:

- I NUMERI DEVONO ESSERE RAPPRESENTATI CON LO **STESSO ESPONENTE** (PORTARE L'ESPONENTE **MINORE** ALLO STESSO GRADO DI QUELLO **MAGIORE**).
- EFFETTUARE LE OPERAZIONI CON LE MANTISSE (INCLUSO L'**HIDDEN BIT**).
- **NORMALIZZARE** IL RISULTATO.
- CONTROLLARE LA PRESENZA DI **OVERFLOW / UNDERFLOW**.
- ARROTONDARE SE **NECESSARIO** (BIT TRONCATI).
- IN CASO **NORMALIZZARE ANCORA**

QUINDI:

VALORE ASSOLUTO

$$|N_1| = 110.1_2 \quad |N_2| = 10.01_2$$

NORMALIZZO: $N_1 = 1.101 \cdot 2^3$ $N_2 = 1.001 \cdot 2^1$

N_2 HA L'ESPONENTE **PICCOLO** QUINDI $N_2 = 0.1001 \cdot 2^3$

ESEGUO

L'OPERAZIONE:

$$N_1 = 1.101000000000000000000000 \cdot 2^3 +$$

$$N_2 = 0.100100000000000000000000 \cdot 2^3 =$$

$$1.001100000000000000000000$$

NORMALIZZO: $1.001100000000000000000000 \cdot 2^3 =$

$$= 1.000110000000000000000000 \cdot 2^3$$

TROVO L'ESPONENTE IN ECCESSO 127:

$$R = 3 + 127 = 130 \rightarrow 10000010$$

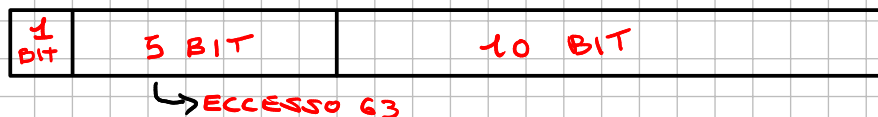
$$1 | 10000010 | 000110000000000000000000$$

Somma in FP 16-bit (mantissa 10 bit):

$$1.6875 + 0.21875$$

- ☐ a. 0 01111 1110100001
- ☒ b. 0 01111 1110100000
- ☐ c. 0 01110 1110100011
- ☐ d. 0 01001 1110100000

CON 16 BIT INTENDIAMO UNA RAPPRESENTAZIONE CON **MEZZA PRECISIONE**:



QUINDI:

$$N_1 = 1.1011_2 \quad N_2 = 0.00111_2 = 1.11 \cdot 2^{-3}$$

$$N_1 = 1.1011 \cdot 2^0 \quad N_2 = 0.00111 \cdot 2^0$$

$$\begin{array}{r} 11 \\ 1.1011000000 \cdot 2^0 + \\ 0.0011100000 \cdot 2^0 = \\ \hline 1.1110100000 \cdot 2^0 \end{array}$$

→ GIÀ **NORMALIZZATO**

$$R = 0 + 63 = 01111_2$$

SEGNO: 0 (POSITIVO)

$$0 | 01111 | 1110100000$$

Semplifica con mappe di Karnaugh in somme di prodotti (l'apostrofo significa negato):

$$F = AB(CD' + CD) + C'D'(AB + A'B) + ABCD$$

☒ a. $Y = ABC + AC'D'$

☐ b. $Y = ABC' + A'CD$

☐ c. $Y = AB + AC$

☐ d. $Y = BC + AD'$

$$F = AB(C\bar{D} + CD) + \bar{C}\bar{D}(AB + A\bar{B}) + ABCD$$

SEMPLIFICHIAMO:

$$F = AB\left(C(\overbrace{\bar{D} + D}^1)\right) + \bar{C}\bar{D}\left(A(\overbrace{\bar{B} + B}^1)\right) + ABCD =$$

$$= \underline{ABC + \bar{C}\bar{D}A + ABCD} = ABC(\underbrace{D + 1}_1) + \bar{C}\bar{D}A = \text{ABC} + A\bar{C}\bar{D}$$

Ci sono arrivato senza mappa ma proviamoci lo stesso:

A	B	C	D	ABCD	$\bar{C}\bar{D}A$	ABC	F
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0
0	0	1	0	0	0	0	0
0	0	1	1	0	0	0	0
0	1	0	0	0	0	0	0
0	1	0	1	0	0	0	0
0	1	1	0	0	0	0	0
0	1	1	1	0	0	0	0
1	0	0	0	0	1	0	1
1	0	0	1	0	0	0	0
1	0	1	0	0	0	0	0
1	0	1	1	0	0	0	0
1	1	0	0	0	1	0	1
1	1	0	1	0	0	0	0
1	1	1	0	0	0	1	1
1	1	1	1	1	0	1	1

ORA CREO LA MAPPA DI KARNAUGH DI 2^4 CELLE VISTO CHE LE RIGHE DELLA FUNZIONE $F=(A,B,C,D)$ SONO 4 ($k=4$)

METTO LE ETICHETTE A DISTANZA DI HAMMING 1 (DEVO CAMBIARE DI 1 BIT).

E OGNI SOTTORETTANGOLO DI $2^{(n-k)}$ CELLE ADIACENTI, SONO ESPRIMIBILI COME IL PRODOTTO DELLE k VARIABILI FISSE, PRESE NEGATE SE IL LORO VALORE È 0, E NON NEGATE SE IL LORO VALORE È 1. (CON PIÙ RETTANGOLI SI FA LA SOMMA DEI PRODOTTI (SP)).

CAMBIAVO DI 1 BIT
(DA 01 → 10)
CAMBIAVO 2 BIT

	AB	00	01	11	10
CD	00			1	1
	01				
	11			1	
	10			1	

$$F = A\bar{C}\bar{D} + ABC$$

POSSO RACCOGLIERE SOTTORETTANGOLI CON POTENZE DI 2.

Presa la funzione $f(a,b,c,d)$ il cui valore è 1 quando il numero rappresentato in binario da $abcd$ è primo o divisibile per 3 determinare una espressione minimale per f come somma di prodotti

- X a. $f(a,b,c,d) = a'c + bd + ad + abc'$
 b. $f(a,b,c,d) = c + bd + ad$
 c. $f(a,b,c,d) = a'c + ad$
 d. $f(a,b,c,d) = ad + abc' + b'd'$

	A	B	C	D	F
0	0	0	0	0	0
1	0	0	0	1	0
2	0	0	1	0	1
3	0	0	1	1	1
4	0	1	0	0	0
5	0	1	0	1	1
6	0	1	1	0	1
7	0	1	1	1	1
8	1	0	0	0	0
9	1	0	0	1	1
10	1	0	1	0	0
11	1	0	1	1	1
12	1	1	0	0	1
13	1	1	0	1	1
14	1	1	1	0	0
15	1	1	1	1	1

EFFETTIVAMENTE LO ZERO È DIVISIBILE MA SE

MESSO A UNO L'ESERCIZIO È SBAGLIATO.

	AB	00	01	11	10
CD	00	0	0	1	0
	01	0	1	1	1
	11	1	1	1	1
	10	1	1	0	0

$$F = AB\bar{C} + AD + BD + \bar{A}C$$